的第一章漫游 1.Hello.c（源程序）预处理 cpp hello.（i 修改了的）编译器 cc1 hello.s（汇编程序）汇编器 as hello.o（可重定位目标程序） 链接器 ld hello（可执行目标程序）2.**文件**是对 **I/O 设备**的抽象表示，**虚拟内存**是对**主存和磁盘 I/O 设备**的抽象表示，**进程**则是对**处理器、主存和 I/O 设备**的抽象表示，在处理器里，**指令集架构(ISA)**提供了对**实际处理器硬件**的抽象，**虚拟机**，它提供对整个计算机的抽象，包括操作系统、处理器和程序 3.Amadahl 定律 **S=1/((1-a)+a/k)** 符号说明：系统执行某程序需要时间 Told 系统某部分所需执行时间与该时间的比例为 α，而该部分性能提升比例为 k，加速比为 S=Told/Tnew 4 堆：代码和数据区后紧随着的是运行时堆。当调用像 malloc 和 free 这样的 C 标准库函数时，堆可以在运行时动态地扩展和收缩。5 栈：用户栈在程序执行期间可以动态地扩展和收缩。特别地，每次我们调用一个函数时，栈就会增长；从一个函数返回时，栈就会收缩。6 超标量：处理器可以达到比一个周期一条指令更快的执行效率 超线程：允许一个 CPU 执行多个控制流的技术gcc -E hello.c -o hello.i gcc -S hello.c -o hello.s gcc -c hello.s -o hello.o gcc hello.o -o hello

第二章信息表示处

1 最低有效字节 LSB 在最前面的方式，称为小端法，：Linux，Window 等。
最高有效字节 MSB 在最前面的方式，称为大端法，使用大端法的机器：Sun
字节=8bit，字长是 CPU 的 ALU 位数或通用寄存器位数，主流是 64 位(8 字节)
2.有符号数和无符号数的转换 处理同样字长:数值可能会改变，但是位模式不变3.64 位各数据类型 char1 short2 int4 long8 long long8 pointer*8 float4 double8
32 位区分 long4 pointer*4
4 位运算 和&, 或 |, 按位取反~, 按位异或^ 均按位运算
逻辑运算 &&两边都真, ||一边为真, ！取反 结果不是 0 就是 1
5.左右移 只需注意右移 逻辑 0 算术最高位
6.扩展 **(例如从 short int 到 int 的转换)**无符号数: 填充 0 有符号数:符号扩展
截断 (例如从 unsigned 到 unsigned short 的转换) 无论有/无符号数:多出的位(高位)均被截 结果重新解读 无符号数: 相当于求模运算 有符号数: 与求模运算相似
7.当执行一个运算时，如果它的一个运算数是有符号的而另一个是无符号的，那么 C 语言会隐式地将**有符号参数强制类型转换为无符号数**，并假设这两个数都是非负的，
8.乘法 x*y 取模，无符号数直接读，有符号数按补码读 U2T,若最高位 1 求的是绝对值
除法 无符号逻辑右移 有符号正数算术右移 负数有符号 $x+2^{移位}-1$ 再右移
9 IEEE 浮点： $V=(-1)^s \times M \times 2^E$ C浮点常数IEEE754编码舍入规则是向偶数舍入
在单精度浮点格式（float)s、exp 和 frac 字段分别为 1 位、k=8 位和 n=23 位，
在双精度浮点格式 double)s、exp 和 frac 字段分别为 1 位、k=11 位和 n=52 位 **数值在趋近于 0 时变密集**
10 规格化 exp 值不全是 0/1 阶码 **E=e-bias** 无符号数读出即可 $Bias=2^{k-1}-1$。
float 的 bias 是 127 double 的 bias 是 1023， $M=1+f$ $(f_{22},f_{21},,,,f_1,f_0)$
11.非规格化 exp 值全为 0 (s=1 正 0，s=0 负 0) **E=1-bias** M=f
12.特殊值 exp 值阶码全为 1，f=0 s=1/0 对应正无穷负无穷，f≠0，NaN
13.整数除法舍入：**向零舍入** 浮点数的舍入：**向偶数舍入**
小数转内存：0.4：x2=0.8→0；x2=1.6→1；x2=1.2→1；x2=0.4→1....0.01100110

手工转换浮点数
e.g.: 将 $(10.2)_{10}$ 转为 f32 （规格化）
$10.2=(1010.0011...)_2=(1.0100011...\times 2^{(3)_{10}})_2$
$s=0$, Bias = 127, $E=3+127=130=(10000010)_2$
• 取frac前 23 位：01000110011001100110011
• 组合：0 10000010 01000110011001100110011
= 0x41233333，内存表示：（小端序存储为 33 33 23 41）

标志位	含义	置 1 时结果满足的条件
ZF	零	结果为 0
SF	负	结果最高位为 1
CF	无符号进位	无符号运算有进位
OF	有符号溢出	有符号运算溢出
PF	偶	最低字节中 1 的个数为偶数

14.double、float 和 int 的转换 从 int 转换成 float，数字不会溢出，但是可能被舍入。从 int 或 float 转换成 double，不会溢出，也不会舍入. 从 double 转换成 float，可能溢出或舍入. 从 float 或者 double 转换成 int，值将会向零舍人，也可能溢出
15.怎样判断两个浮点数是否相等? 由于浮点数的 ieee754 编码表示存在着精度、舍入、溢出、类型不匹配等问题，两个浮点数不能够直接比较大小，应计算两个浮点数的差的绝对值，当绝对值小于某个可以接受的数值精度时认为相等. 不要轻易进行浮点数比较可能多个数用一个编码，都相等，多个编码不一致的数，但在精度范围内都相等。
16.程序中的 2 进制、10 进制、16 进制数，在**汇编阶段**变成 2 进制。C 语言程序中的整数常量、整数常量表达式是在 **编译阶段** 变成 2 进制补码的，C 语句中的有符号常数，在 **编译阶段** 转换成了补码，C 语言程序中的常量表达式的计算是由 **编译器** 完成的，十进制的有符号常数由 **编译器** 转换为二进制补码，C 语言的常量表达式的计算是由 **编译器** 完成的
17.左移无论有/无符号数，都可用逻辑左移实现乘以 2 的 k 次方 右移无符号数: 逻辑右移,除以 2^k(除法 +向 0 舍入) 有符号数: 算术右移 正整数:除以 2^k(除法 + 向 0 舍入) 负整数:除以 2^k(除法 + 远离 0 舍入)，使用偏置来修正 $\frac{x+2^k-1}{2^k}$
18 **原码：** 最高位为符号位：0 表示正数，1 表示负数，其余位表示数值的绝对值。（直观易懂 容易理解 但是存在双 0）
反码： 负数原码各个位置取反，符号位不变 （存在双 0 硬件实现不简洁）
补码： 正数与原码相同，负数该数的反码加 1（唯一 0 表示，有运算优势，扩展方便）
移码： 在补码的基础上加一个偏移量，通常是 2 的 n-1 次方 （n 位）（便于比较大小，适合浮点数运算）
19 汉字 UTF-8 编码的核心特性 **优点** 空间高效：常用汉字仅占 3 字节，生僻字扩展为 4 字节，比固定 4 字节的 UTF-32 更节省存储空间；
容错性强： 每个字符的字节标识明确，可快速定位字符边界，即使部分数据损坏也不影响整体解析；**跨平台兼容：** 统一了 Linux、Windows、互联网的编码标准，减少跨系统文件拷贝时的乱码问题。**局限性** 字节数不一致：与英文字符的 1 字节、GBK 汉字的 2 字节相比，UTF-8 汉字字节数更多，会增加部分场景的存储和处理开销；**排序复杂：** UTF-8 编码顺序基于 Unicode 码位，与汉字的拼音顺序、笔画顺序均不对应，直接按编码排序会出现逻辑混乱；**旧系统兼容问题：** Win10 及以前的记事本、VS 等默认使用 GBK 编码，与 UTF-8 文件直接互通会产生乱码，需手动转换。

三 机器级表示

1 摩尔定律：处理器晶体管数量、性能持续提升
2 寄存器
```
%rax %eax %ax %al 返回值     ; %rbp %ebp %bp %bpl 栈帧
%rbx %ebx %bx %bl 基址寄存器 ; %rsp %esp %sp %spl 栈指针
%rcx %ecx %cx %cl 第 4 个参数 ; %r8 %r8d %r8w %r8b 第 5 个参数
%rdx %edx %dx %dl 第 3 个参数 ; %r9 %r9d %r9w %r9b 第 6 个参数
%rsi %esi %si %sil 第 2 个参数
%rdi %edi %di %dil 第 1 个参数 （64，32，16，8）
```
3 寻址模式以及数据传送指令
立即数：用来表示常数值，是$后面跟一个标准 C 表示法表示的整数
寄存器：ra 表示任意寄存器 a，R[ra]来表示它的值，寄存器集合看成一个数组 R，用寄存器标识符作为索引。
存储器（内存引用）：根据计算出来的地址访问某个内存位置。Mb[Addr]表示对存储在内存中从地址 Addr 开始的 b 个字节值的引用，
$Addr = Imm(r_b, r_i, s)=Imm+R[r_b]+R[r_i]*s$

类型	格式	操作数值	名称
立即数	$Imm	Imm	立即数寻址
寄存器	r_a	$R[r_a]$	寄存器寻址
存储器	Imm	M[Imm]	绝对寻址
存储器	(r_a)	$M[R[r_a]]$	间接寻址
存储器	$Imm(r_b)$	$M[Imm+R[r_b]]$	（基址 + 偏移量）寻址
存储器	(r_b, r_i)	$M[R[r_b]+R[r_i]]$	变址寻址
存储器	$Imm(r_b, r_i)$	$M[Imm+R[r_b]+R[r_i]]$	变址寻址
存储器	$(,r_i,s)$	$M[R[r_i]\cdot s]$	比例变址寻址
存储器	$Imm(,r_i,s)$	$M[Imm+R[r_i]\cdot s]$	比例变址寻址
存储器	(r_b,r_i,s)	$M[R[r_b]+R[r_i]\cdot s]$	比例变址寻址
存储器	$Imm(r_b,r_i,s)$	$M[Imm+R[r_b]+R[r_i]\cdot s]$	比例变址寻址

图 3-3 操作数格式。操作数可以表示立即数(常数)值、寄存器值或是来自内存的值。比例因子 s 必须是 1、2、4 或者 8

4 movb 传送字节 w 字 l 双字 q 四字 源操作数和目的操作数大小一致 movl 指令以寄存器作为目的时。会把寄存器的高位 4 字节设置为 0
常规 movq 指令只能以表示为 32 位补码数字的立即数作为源操作数，然后把这个值符号扩展为 64 位，movabsq 能够以任意 64 位立即数值作为源操作数，并且只能以寄存器作为目的
若将较小源值复制到较大的目的时，有 movz（零扩展）和 movs（符号扩展）
movzbw movzbl movzbq movzwl movzwq 第五个字母是源 第六个是目的
movsbw movsbl movsbq movswl movswq movslq
cltq 仅限于将%eax 符号扩展到%rax
5 8086 有 4 个 16 位段寄存器 CS（代码段）指明代码段的起始地址 SS（堆栈段）指明堆栈段的起始地址 DS（数据段）指明数据段的起始地址 ES（附加段）指明附加段的起始地址
6 栈操作 栈顶是低地址 栈由高地址指向低地址
```
pushq 入栈 pushq %rax     等价于两条指令的组合：
subq $8, %rsp      ; 栈指针%rsp 减 8（栈向下增长，指向新栈顶）
movq %rax, (%rsp); 将%rax 的值写入新栈顶地址
popq %rdx     等价于两条指令的组合：
movq (%rsp), %rdx ; 从栈顶地址读取值到%rdx
addq $8, %rsp      ; 栈指针%rsp 加 8（指向原栈顶的上一个位置）
```
x86-64 中栈与程序代码、数据共享同一内存空间，因此可以通过内存寻址直接访问栈内任意元素 栈有保存的功能
7 算术和逻辑操作

指令	效果	描述
leaq S, D	D ← &S	加载有效地址
INC D	D ← D + 1	加 1
DEC D	D ← D − 1	减 1
NEG D	D ← −D	取负
NOT D	D ← ~D	取补
ADD S, D	D ← D + S	加
SUB S, D	D ← D − S	减
IMUL S, D	D ← D * S	乘
XOR S, D	D ← D ^ S	异或
OR S, D	D ← D | S	或
AND S, D	D ← D & S	与
SAL k, D	D ← D << k	左移
SHL k, D	D ← D << k	左移（等同于SAL）
SAR k, D	$D \leftarrow D >>_A k$	算术右移
SHR k, D	$D \leftarrow D >>_L k$	逻辑右移

8 加载有效地址 leaq 该指令并不是从指定的位置读入数据，而是将有效地址写入到目的操作数 目的操作数必须是寄存器 heyiw
可以用作加法和部分乘法 eg: x in %rdi y in %rsi
leaq （%rdi，%rsi，4） = x + 4y
9 条件码 CF：进位标志:无符号操作的溢出 ZF：零标志:最近操作结果为 0 SF：符号标志—最近操作结果为负数 OF：溢出标志—导致一个补码溢出
10 CMP（cmpb、w、l、q）:类似 SUB（右减左）指令，
TEST（testb，w，l，q）类似于 AND（和） 置条件码

指令	同义名	效果	设置条件
sete D	setz	D ← ZF	相等/零
setne D	setnz	D ← ~ZF	不等/非零
sets D		D ← SF	负数
setns D		D ← ~SF	非负数
setg D	setnle	D ← ~(SF ^ OF) & ~ZF	大于（有符号>）
setge D	setnl	D ← ~(SF ^ OF)	大于等于（有符号>=）
setl D	setnge	D ← SF ^ OF	小于（有符号<）
setle D	setng	D ← (SF ^ OF) | ZF	小于等于（有符号<=）
seta D	setnbe	D ← ~CF & ~ZF	超过（无符号>）
setae D	setnb	D ← ~CF	超过或相等（无符号>=）
setb D	setnae	D ← CF	低于（无符号<）
setbe D	setna	D ← CF | ZF	低于或相等（无符号<=）

11 set 指令 setx D 如果满足 x，则 D=1，不满足则 D=0
对于 x： e 相等 ne 不相等 s 负数 ns 非负数
g 有符号大于 ge 有符号大于等于 l 有符号小于 le 有符号小于等于
a 无符号的大于 ae 无符号大于等于 b 无符号小于 be 无符号小于等于
12 jmp 直接跳转 j+x（x 同 11 中列出） 如果满足 x 则跳转，不满足则不
13 cmov +x S,R 当满足 x 时，指令把源值 S 复制到目的 R x 同 11
14 switch 与跳转表
switch 语句用于基于一个整数表达式（如 int、char、枚举类型）的值，跳转到对应的 case 分支执行，语法上包含 switch(表达式)，多个 case 常量分支和可选的 default 分支。它相比多个 if-else if 结构，在分支较多且常量值连续时，可读性和执行效率更高。当 switch 的 case 常量连续且数量较多时，编译器会将各个 case 的处理代码入口地址存入一个数组（即跳转表）。执行时，先计算表达式的值与最小 case 值的偏移量，再直接通过偏移量索引跳转表，获取对应入口地址并跳转执行，时间复杂度为 O(1)。使用跳转表的优点是：执行开关语句的时间与开关情况的数量无关
15 过程的本质：是软件中封装代码的抽象机制，通过参数传递、执行逻辑、返回值完成特定功能，对应高级语言的函数、方法、子例程等，x86-64 下通过栈（栈帧）和寄存器实现调用与返回。
过程调用的核心动作：调用者 P 调用被调用者 Q 时，需完成控制传递（跳转到 Q 的代码起始地址）、数据传递（传参、返回值）、内存分配与释放（为 Q 分配栈帧，返回后释放）。
运行时栈（Runtime Stack）栈是 x86-64 实现过程调用的核心数据结构，遵循后进先出（LIFO）原则：
栈帧（Stack Frame）：每个活跃的过程对应一块独立的栈内存区域，称为栈帧，包含：参数区：调用者传递给被调用者的参数（超出 6 个整型参数时存于此）；**返回地址：** 调用者中 call 指令的下一条指令地址，用于被调用者返回；**被保存的寄存器：** 被调用者需要保留的寄存器值（如%rbx、%rbp）；**局部变量区：** 过程的局部变量（寄存器存不下时）；
参数构造区： 为调用其他过程准备的参数空间。（3）栈帧的生命周期：过程执行时创建栈帧，返回时释放，栈指针恢复到调用前的位置。
控制转移（过程调用与返回）
控制转移的核心是程序计数器（PC）的修改，通过 call 和 ret 指令实现：
1. **call 指令：** 功能：① 将返回地址（call 下一条指令的地址）压入栈；② 将 PC 设置为被调用过程的起始地址，跳转到目标过程。
格式：callq Label （直接调用）或 callq *Operand （间接调用）。
2. **ret 指令：** 功能：从栈顶弹出返回地址，将 PC 设置为该地址，回到调用者的执行流。格式：retq（返回指令）。**3. 执行流程示例：** 调用者 main 执行 callq top→压入 main 的返回地址→PC 跳转到 top 执行 → top 执行 callq leaf→压入 top 的返回地址→PC 跳转到 leaf 执行 → leaf 执行 retq→弹出返回地址回到 top→top 执行 retq→ 弹出返回地址回到 main
参数传递整型/指针参数前 6 个参数分别存%rdi、%rsi、%rdx、%rcx、%r8、%r9 超出 6 个的参数，从第 7 个开始按逆序压入栈（参数构造区）
16 可能引起缓冲区溢出的函数有：strcpy()、sprintf()、gets()、scanf()、strcat()，这些函数不需要告诉它们目标缓冲区的大小，就产生一个字节序列：gets()函数无法确定是否为保存整个字符串分配了足够的空间，strcpy, strcat：任意长度字符串的拷贝，scanf, fscanf，sscanf 使用%s 转换符时
17 对抗缓冲区溢出攻击的措施（防）
（1）避免溢出漏洞（函数、代码细节）如用 fgets 代替 gets、strncpy 代替 strcpy，scanf 函数中别用%s（或用%ns 代替%s，其中 n 是一个合适的整数）
（2）使用系统级的防护 1 栈随机化（随机的栈偏移），程序启动后，在栈中分配随机数量的空间；2 限制可执行代码区域（非可执行代码段），消除攻击者向系统中插入可执行代码的能力 3 栈破坏检测（栈金丝雀）

四 处理器体系结构

层次结构：高级语言→汇编语言→操作系统/ISA→体系结构(流水线)→逻辑门/电路。
1 ISA 指令集体系结构 是软硬件系统的交界面
2 Y86-64 指令集体系结构 **程序员可见状态：** 15 个程序寄存器(除了%r15)，每个 64 位 3 个条件码:ZF、SF、OF 程序计数器:保存下一条指令地址 内存:虚拟地址空间 状态码（Stat）：AOK（正常）、HLT（停止）、ADR（地址错误）、INS（非法指令）
CISC(复杂指令集，x86)指令多且长度不等，有些指令执行很慢
RISC(精简，ARM)指令少且定长，只通过 load/store 访存，计算都在寄存器
Y86-64 是 CISC 的指令子集，但设计风格带有 RISC 的某些简化特征
3 ISA 指令格式的选择应遵循的几条基本原则：应尽量短；要有足够的操作码位数；指令编码必须有唯一的解释，否则是不合法的指令；指令字长应是字节的整数倍；合理地选择地址字段的个数；指令尽量规整 注意：是小端法储存地址,从右往左
指令编码格式： 操作码（指令码+功能码+源寄存器编码+目的寄存器编码(1-10 字节)
4 指令种类与用法 ①数据传送指令 **i r movq V, rB:** 将立即数 V 送入寄存器 rB。
r r movq rA, rB: 寄存器 rA 传给 rB **r m movq rA, D(rB):** 将寄存器 rA 的值写入内存
m r movq D(rA), rB: 从内存（地址为 D+rA 的值）读到寄存器 rB
②整数操作指令(只能寄存器到寄存器) **addq rA,rB** 将 rA 的值加到 rB 中
subq rA,rB： rB 减 rA，覆盖 rB 的值 **andq rA, rB：** 按位与 **xorq rA, rB：** 按位异或
③跳转指令 **jmp Dest**:无条件跳转 **jXX Dest**:有条件跳转 **cmovXX rA,rB**:条件传送指令
④栈操作 **pushq rA：** 压栈：%rsp 减 8，将 rA 保存的字储存到内存%rsp 中
popq rA： 出栈：从内存的%rsp 位置读取字存放在 rA 中，%rsp 加 8
⑤子程序调用返回 **call Dest**:下条指令的地址入栈，开始执行位于 Dest 的指令
ret： 从栈中弹出值用作下一条指令的地址 (Dest 是目标指令的地址)
nop： 不做任何事。。 **halt：** 停止执行指令，终止模拟器(stat 设为 HLT)
5 CPU 硬件组件：**组合逻辑硬件：** 算术逻辑单元(ALU)和多路复用器(MUX)构成,不存储信息，输出仅取决于当前输入，在 HCL 中表现为一系列布尔表达式
存储元件： *时钟寄存器*：如程序计数器、条件码(CC)、状态码(stat)，只有在时钟从 0 变 1 的瞬间触发，存入寄存器变成新的输出 *随机存取储存器*(RAM):如寄存器文件(存%rax,rbx，可以读两个写一个)，数据内存(存真正的程序数据)
6 SEQ 处理器的特点 每时钟周期完成一条完整的指令(6 个阶段)(所以周期必须设长)
7 指令执行流程 (icodeifun 指令代码和功能，valP 是下条指令地址，valC 常数/偏移量)
取指 (Fetch)：根据 PC 的值从内存读取指令字节，产出 rA,rB, icode:ifun,valC，valP
译码 (Decode)：从寄存器文件读入操作数。产出 valA 和 valB(寄存器的数值)
执行 (Execute)：ALU 执行运算、检查条件码或计算有效地址。产出中间值 valE
访存 (Memory)：从内存读写数据。产出 valM(如果读内存)
写回 (Write back)：将结果 valE 或 valM 写回到*寄存器文件*。
更新 PC(PC update)：将 PC 设置为下一条指令的地址。
压栈举例：pushq rA ①执行：valE=valB-8 (valA=要压栈的寄存器数值，valB 是栈指针)
②访存：M[valE]=valA (值存入新栈顶) ③写回：R[%rsp]=valE(更新栈指针)
8 流水线通用原理 **核心思想：** 通过并行处理，提高吞吐量(单位时间完成指令数量)
需要解决的问题：1.数据冒险： 后面指令需要前面指令还没来得及写回寄存器的结果
解决： 转发/旁路：直接从执行/访存阶段通过导线把结果"传"给后面的指令，不等写回。暂停：如果转发解决不了（如 Load 指令），就让后面的指令停一拍。
2 控制冒险： 遇到跳转指令，不知道该取哪里的指令 **解决：** 分支预测：先"猜"一个路径走。冲刷 (Flush)：如果猜错了，就把流水线里错误的指令清除，重新取指
9 基于 ISA 的软件生态建设**编译器：** 将高级语言（C）翻译成特定 ISA 的汇编代码。**汇编器：** 将汇编代码转换为该 ISA 定义的二进制机器码；**模拟器：** 用软件代码模拟 ISA 的逻辑；**操作系统 (OS)：** 利用 ISA 提供的特权指令管理硬件资源；**应用软件：** 最终运行在操作系统之上，间接通过 ISA 调用硬件计算能力。 鬼知道这是啥?
写出 mrmovq D(rB), rA 指令在各阶段的操作 。
取指：icode:ifun ← M1[PC]; rA:rB ← M1[PC+1]; valC ← M8[PC+2]; valP ← PC+10 。
译码：valB ← R[rB] 。 执行：valE ← valB + valC 。 访存：valM ← M8[valE] 。
写回：R[rA] ← valM 。更新 PC：PC ← valP

五 优化

1 一般有用的优化:1 **代码移动**(将重复计算结果不变的表达式移出循环,减少执行频率)
for (int i = 0; i < strlen(s); i++) → int len = strlen(s); for (int i = 0; i < len; i++) {...}
2 **复杂运算简化**(移位、加，替代乘法/除法) x*16 → x<<4 循环 16*i → t+=16
3 **共享公用子表达式**(重复子表达式存入变量，-O1)减少对内存的读写
4 **函数调用**(去掉不必要的过程函数调用) get_vec_element(v, i, &val) sum += val
移出循环 long *date = get_vec_start(v), sum += data[i]避免栈帧创建和跳转开销
2 面向编译器的优化方法 1 采用内联函数 2 采用局部变量 上述都是编译器自动优化：1 常数折叠 2 死代码消除 3 寄存器分配 4 运算化简
优化编译器的局限性 提供程序到机器间的有效映射；（通常）不提高渐进效率；难以克服优化障碍（潜在的内存别名使用，的函数副作用）；在基本约束条件下运行；大多数分析只在过程中执行；基于静态信息的；当有疑问时，编译器必须是保守的 程序性能度量标准——每个元素的周期数；性能比渐进复杂度更重要，**常数因子**也很重要
3 妨碍优化的因素 1 **储存器别名使用** 两个指针指向同一个内存地址，编译器不知 u 到函数什么时候被调用，只会不断的读写，对策：引入局部变量，存储中间结果，减少内存读写 2 **函数调用** 函数可能有副作用(调用改变全局变量，返回不同值)，编译器不将循环内的函数移出循环 对策：手动将结果不变的函数 strlen(s)移出循环
4 指令级并行现代处理器结构 **超标量**一个周期执行多个操作 **乱序执行 功能 单元：** 如浮点加法器、乘法器，有**延迟**(执行运算所需时钟周期)和**发射时间**(两次运算间隔最小周期数) **容量：** 能够执行某个运算的功能单元数量**延迟界限：** 任何必须按照严格顺序完成合并运算的函数所需要的最小 CPE 值**关键路径：** 程序限制速度的最长逻辑路径
吞吐量界限： 发射间隔/该类运算单元的数量 延迟可以无限逼近吞吐量极限
5 循环展开 通过增加每次循环处理的元素个数，减少循环开销(计数器更新分支跳转)
带重组的展开 acc = (acc OP data[i]) OP data[i+1];→acc = acc OP (data[i] OP data[i+1])
先把两个 date 数据结合，提前计算结果，打破对 acc 的等待 (关键路径减少)
6 提高并行性：**多个累计变量**，拆分 acc0，acc1 **重新结合变换**，改变计算顺序
7 寄存器溢出：循环展开倍数（K）是不是越大越好? CPU 寄存数量有限，装不下方到内存(栈)里，性能曲线先下降(快)后大幅回升(慢) 567 是面向指令级并行(CPU)优化
8 面向存储器优化： 局部性原理、循环交换（步长为 1 访问）、矩阵分块
块内所需的数据总量 < Cache 容量 块大小用 B/数据类型字节大小
```
分块：for (i = 0; i < n; i++)        for (j = 0; j < n; j++)        B[j][i] = A[i][j];
优化：int bsize = 8; int v1, v2, v3, v4, v5, v6, v7, v8;
for (ii = 0; ii < n; ii += bsize) {for (jj = 0; jj < n; jj += bsize) {
for (i = ii; i < ii + bsize; i++) { 内层 for (j = jj; j < jj + bsize; j++) {
v1 = A[i][ii];    v2 = A[i][ii+1];......    B[ii][i]    = v1; B[ii+1][i] = v2;......    B[j][i] = A[i][j];
```
9 条件分支预测 引导编译器翻译出条件数据传送指令 int add_val = (val > 128) ? val : 0;
```
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
        for (k = 0; k < M; k++)
            sum += a[k][i][j]; //k 在最内层，导致跨行访问
优化：int limit = M;
float acc0 = 0, acc1 = 0; // 优化 1：使用多个累加器（面向 CPU）
for (k = 0; k < M; k++)    // 优化 2：将 k 移到最外层（面向 Cache，步长为 1 访问）
    for (i = 0; i < N; i++)
        for (j = 0; j < N - 1; j += 2) { // 优化 3：循环展开
            acc0 += a[k][i][j];
            acc1 += a[k][i][j+1];}
sum = acc0 + acc1;
②int fibr(int n) {        if (n == 0 || n == 1) return 1;
    return fibr(n-1) + fibr(n-2); // 指数级递归、效率极低易溢出
优化：double fib_optimized(int n) {
    double x = 1, y = 1, z;        int i;        if (n <= 1) return 1;
    for (i = 1; i < n; i++) {        z = x + y;    x = y;    y = z;}    return y;}
```
或循环展开 for (i = 1; i < n - 5; i += 5) {z = 5 * x + 8 * y; x = 3 * x + 5 * y; y = z;} 加上方补尾

六储存器 1.分类 按工作性质/存取方式分类
随机存取存储器(RAM) 每个单元读写时间一样，且与各单元所在位置无关如：**内存**。
顺序存取存储器 (SAM)数据按顺序从存储载体的始端读出或写入，因而存取时间的长短与信息所在位置有关。例如：**磁带**。
直接存取存储器(DAM)直接定位到读写数据块，在读写数据块时按顺序进行。如**磁盘**。
相联存储器（AM） (CAM)按内容检索到存储位置进行读写。例如：**快表**。
按功能/容量/速度/所在位置分类 寄存器(Register)封装在 CPU 内，用于存放当前正在执行的指令和使用的数据，用触发器实现，速度快，容量小（几~几十个）
高速缓存(Cache)位于 CPU 内部或附近，用来存放当前要执行的局部程序段和数据用 SRAM 实现，速度可与 CPU 匹配，容量小（几 MB）
主存(内存) 位于 CPU 之外、主板之上，由 DRAM 构成，存取速度较快、容量较大但成本相对较高。它直接与 CPU 相连，CPU 可以通过地址总线和数据总线对主存进行随机读写操作。**易失性存储器**，断电后内容会丢失，主要用于临时存放正在运行的程序及其所需的数据，是程序运行的工作区域。
辅存(外存)位于主机之外，包括硬盘、U 盘、光盘等，容量大、成本低但存取速度较慢。辅存不与 CPU 直接相连，CPU 无法直接访问其中的数据，必须先通过 I/O 操作将数据从辅存调入主存后才能使用。**非易失性存储器**，断电后数据不会丢失，主要用于长期保存暂不运行的程序、大量数据和各种文件。磁盘 1GB=109 B TB12
SRAM(cache)每个位六个晶体管双稳态 DRAM(内存)每个位一电容一个晶体管
2.主存中存放的是什么信息? CPU 何时会访问主存?
指令及其数据！CPU 执行指令时需要取指令、取数据、存数据！
3.地址译码器的输入是什么? 输出是什么? 可寻址范围多少?
输入是地址，输出是地址驱动信号（只有一根地址驱动线被选中）。
可寻址范围为 0~236-1，即主存地址空间为 64GB（按字节编址时）。
4.磁盘访问时间访问时间 = 寻道时间 + 平均旋转延迟 + 数据传输时间
通常寻道时间为：3—9 ms；旋转时间为一圈的时间（/2 平均）；传送时间一圈/扇区
5.逻辑块与物理扇区之间的映射关系
由磁盘控制器维护；磁盘控制器将逻辑块号转换为一个三元组(盘面,磁道,扇区)
6.固态硬盘 page(写入 1-0)-block(擦除 0-1)-plane-die-flahchip
7.时间局部性:当前被访问的信息近期很可能还会被再次访问
空间:在最近的将来用到的信息空间地址上是临近的(顺序访问)二维数组行优先存储
如何编写局部性好的程序? 步长为 1 的访问；尽可能重用寄存器中的变量
8.软硬件的基本稳定特性:高速存储器技术成本高，容量小，且耗电大，易发热；CPU 与存储器之间的速度差距越来越大；编写良好的程序往往表现出良好的局部性
9.**高速缓存** Cache(S,E,B,m)-set-E 行-(valid-Tag-Bbyte) m-地址是四位 C=S*E*B
地址： t bits(标记)-s bits(行/组索引)-b bits(块 bias)

(gdb) print $eax $RIP (gdb) info reg 打印当前CPU所有通用寄存器的内容
(gdb) i proc all看本进程所有信息 (gdb) i proc map看内存(gdb) bt 看调用栈
GDB 调试常用命令 break <func>: 设置断点 run <args>: 运行程序并传参 n/s: 单步执行(不/进函数) x/<fmt> <addr>: 查看内存，fmt格式如8x c: 继续执行直到下一个断点 p <var>: 打印变量值 i reg: 查看寄存器状态 i proc map: 查看内存映射 layout asm: 查看汇编代码布局

e.g.: (i32) 0x1A3CFF12

	小端序	大端序	
低地址	12 FF 3C 1A	1A 3C FF 12	高地址

进程上下文切换发生在：时间片用尽；外部硬件中断（高优先级进程抢占）；当前进程调用系统调用。
切换阶段（流程）：1.触发中断/系统调用。CPU自动保存当前进程现场到内核栈。2.内核进入调度相关代码，执行switch_to汇编函数保存旧进程所有寄存器。3.调度器选择下一个运行进程，调度器函数schedule()运行，标记旧进程状态。4.加载新进程上下文（核心切换），仍在switch_to汇编，恢复next进程数据：读取next进程PCB里存储的所有寄存器，恢复到CPU寄存器；更换页表，把CR3寄存器写入新进程的PPN，使两个进程地址空间完全隔离；恢复浮点寄存器、调试寄存器；切换内核栈为新进程的内核栈。5.退出内核态，回到用户空间执行新进程，CPU开始从新进程上次暂停的指令地址继续运行。6.刷新TLB缓存，旧虚拟页映射失效CPU淘汰TLB表项；统计调度信息；处理待递达的信号

面向编译器优化
消除内存别名使用：使restrict声明指针不重叠或局部变量存储中间结果算完再写回
内联函数避免编译器因无法分析副作用而不进行优化
消除低效率循环；减少过程调用；消除不必要内存引用

超标量 CPU 优化
带累加器循环展开: 减少分支预测开销，增加指令级并行(打破顺序执行限制)
多个累积变量：利用更多运算单元同时工作

向量处理器优化
向量化：SIMD 指令(AVX 等)，一次处理多个数据
数据对齐：确保以向量宽度对齐，加速向量加载和存储

Cache 优化
优先访问多维数组的行: 利用空间局部性
分块：提高列元素的时间局部性，避免缓存抖动提高命中率

int a[100]; 在作为全局变量、局部变量、参数时是怎么实现对a[i]元素的访问的全局变量：利用%rip+偏移量访问 局部变量：利用%rbp+偏移量访问 参数：先通过%rbp+偏移量得到栈中作为参数传递的a的首地址，然后利用此地址+偏移量访问

静态局部变量是怎么生成汇编语言的空间：全局区 赋初值：定义该变量的函数第一次被调用时 生命周期：整个源程序的周期分析全局变量、局部变量、传值的参数的生命周期静态全局变量：整个源程序的周期 动态全局变量：整个源程序的周期 静态局部变量：整个源程序的周期 动态局部变量：定义它的函数的周期 传值的参数：定义它的函数的周期简述缓冲器溢出的原理缓冲区溢出，简单的说就是计算机对接收的输入数据没有进行有效的检测，向缓冲区内填充数据时超过了缓冲区本身的容量，而导致数据溢出到被分配空间之外的内存空间，使得溢出的数据覆盖了其他内存空间的数据。怎么攻击缓冲器溢出的漏洞？攻击原理：向程序输入缓冲区写入特定的数据，例如在gets读入字符串时，使位于栈中的缓冲区数据溢出，用特定的内容覆盖栈中的内容，例如函数返回地址等，使得程序在读入字符串，结束函数gets从栈中读取返回地址时，错误地返回到特定的位置，执行特定的代码，达到攻击的目的。怎么防范缓冲器溢出漏洞？防范方法：1 代码中避免溢出漏洞：例如使用限制字符串长度的库函数 2随机栈偏移：程序启动后，在栈中分配随机数量的空间，将移动整个程序使用的栈空间地址 3限制可执行代码的区域 4进行栈破坏检查结构体成员作为参数传输/返回是怎么实现的？传递/返回结构体基地址+偏移量处的对应成员

进程 子进程必须由父进程回收，否则成为僵尸进程 errno:记录错误码的外部全局变量，系统调用失败时设置，使用strerror(errno)获得错误原因 僵尸进程：子进程结束后父进程未调用wait(pid)回收，保留进程表项但资源已经释放 孤儿进程：父进程结束后子进程被init收养，若子进程结束则孙进程仍然被init收养
调度 进程上下文包括：寄存器，一级页表**指针**(寄存器)，进程表，文件表(描述符)，不包含全局变量等内存中的数据 时间片轮转调度: 每个进程分配固定时间片，时间片用完则切换到下一个进程 上下文切换：1)保存当前进程的上下文，2)恢复某个先前被抢占的进程被保存的上下文，3)将控制传递给这个新恢复的进程
pid_t fork()创建父进程的副本作为子进程，调用一次父子各返回一次，子进程返回 0，父进程返回子进程 PID，继承文件描述符
int execve(*filename, *argv[], *envp[]) 不返回 filename: 可执行文件路径 argv[]: 参数数组，argv[0]为程序名 envp[]: 环境变量数组，可传全局变量environ 其用新程序替换当前进程映像，加载新程序到当前进程空间并运行，保留 PID 和文件描述符
int waitpid(pid, int *status, int options) wait(): 简化版，等同于waitpid(-1, status, 0) pid: 待回收的子进程 PID，-1表示回收任意子进程 status: 返回子进程退出状态，调用者传入缓冲区 options: 怎么等待，0表示阻塞等待，
其他选项:**pid_t getpid()**获取当前进程的 PID**pid_t getppid()**获取当前进程的父进程 PID**pid_t getpgrp()**获取当前进程所属的进程组 ID

有特殊用途的寄存器
%rax: 存放函数返回值；系统调用号传参; %rsp: 栈指针，指向当前栈顶位置;%rbp: 基址指针，指向当前栈帧底部;%rip: 指向下一条指令的地址，call和ret会修改它
函数参数传递寄存器
64 位程序: 按顺序依次用%rdi, %rsi, %rdx, %rcx(系统调用使用%r10), %r8, %r9，之后参数用栈传递 32 位程序: 全部用栈传递(Linux x86-32)

lea 指令的特殊用法
lea指令可以用来进行某些简单算术运算，效率更高：
lea (%base, %index, scale), %dst意思是计算%base + %index * scale并存入%dst
加法: lea (%src1, %src2, 1), %dst 计算 %src1 + %src2
乘法: lea (%src, %src, n), %dst 计算 %src * (n + 1)，n只能是 1、2、4、8
地址计算: lea 8(%base), %dst 计算 %base + 8

直接映射：组选择-s行匹配v=1,tag 是否一致);字抽取(偏移 b 个 ，B 的后 b 位)

组相连 同上 **全相联**：一个 set 包含所有行 地址直接 tag-b bits 到

10.关于怎么写? 写命中：*直写*（写入存储器） 写回：只写位 cache，需要一个修改写不命中：写分配：内存加载到 cache *非写分配*：直接写到主存

11.为何不用高位来索引高速缓存？，每个连续的数组片(chunk)由 2^t 块组成，这里 t 是标记位数。因此，数组头 2t 个连续的块都会映射到组 0，接下来的 2t 个块会映射到组 1，用高位做索引不能充分利用高速缓存。

12.面向cache的两种优化方法 1.让通用或共享的功能或函数一最常见情况运行得快，专注在核心函数和内循环上 2.尽量减少每个循环内部的缓存不命中数量，反复引用变量是好的(时间局部性)—寄存器-编译器，步长为 1 的 参考模式是好的(空间局部性)—缓存是连续块

13.读命吐量:(读带宽)程序运行时 每秒从存储系统中读取的字节数(MB/s)

14.存储器山：测量读取存吐量作为空间和时间局部性的函数通过二维函数(图-存储器 山 描述存储系统性能 X轴（size 轴）：工作集大小（字节），代表程序访问的数据量，从 KB 级（L1 缓存）到 MB/GB 级（主存）递增,反应**时间局部性**

-Y 轴（stride 轴）：访问步长（字节），代表连续访问数据时的地址间隔，体现**空间局部**性（步长越小，空间局部性越好）。

-Z 轴（吞吐量轴）：读存吐量（MB/s），代表存储系统的读取速度，是性能的核心指标，数值越高代表性能越好。

时间局部性：同一数据被反复访问时，会驻留在缓存中，对应存储器山的“山峰”区域，吞吐量高。

- 空间局部性：步长越小（如步长=1 字节），CPU 能利用缓存行预取数据，空间局部性越好，对应山体“陡峭的上坡”；步长增大（如步长=8/16 字节），空间局部性变差，吞吐量沿“斜坡”下降

第七章链接 1.为什么用链接器：1.模块化 ；2 效率 时间：分开编译（更改源文件不需要重新编译其他源文件）空间：库（可以将公共函数聚合为单个文件，且可执行文件和运行内存映像只包含它们实际使用的函数的代码）

2.链接器作用 1.符号解析:将每个符号引用与一个符号定义关联起来(全局/静态变量，函数名是，局部变量和参数不是符号)2.**符号**重定位:将多个单独的代码节和数据节合并为单个节，并修改地址：把符号从.o 文件的相对位置移动到可执行文件最终绝对内存位置，并更新所有引用，使其指向新地址。

3 可重定位目标 ELF 头(字大小、字节顺序、文件类型，机器类型)程序头表(页面大小，虚拟地址内存段/节，段大小);

.text 节(代码)编译后的机器指令（二进制代码）

.rodata 节(只读数据)如 printf 中的格式化字符串（"Hello%d\n"）、const 修饰的全局变量、switch 语句的跳转表

;.data 节(数据/可读写)已初始化且初值不为 0 的全局变量和静态变量（static）。

.bss 未初始化或初值为 0 的全局变量和静态变量

不装入内存的：.symtab(符号表存储程序中定义和引用的函数、全局变量的符号信息（名称、位置、大小）。).rel.text(重定位信息).debug 节头表

程序运行才产生的:

Runtime Heap(运行时堆)存储内容：程序运行过程中动态分配的内存。调用 malloc

User Stack(用户栈)局部变量：函数内部定义的非静态变量，未初始化的局部变量初始化为 0 的局部变量

4 **符号**：全局符号：由模块 m 定义，能被其他模块引用的符号（如非静态函数和非静态全局变量）。外部符号：由其他模块定义，但在模块 m 中被引用的符号。本地/局部符号：静态属性的 C 函数和全局变量。它们对外部不可见。 本地链接符号不是**普通全局变量：全局符号 静态局部/全局变量：本地符号 普通局部变量：不是符号 静态性：不再存放在栈上，而是存放在数据段 局部性：依然只有这个函数能访问它。**

5 符号解析强：函数和初始化 ***全局变量*** 弱:未初始化 ***全局变量***(int x; 含 0)

规则 1 不允许多个同名的强符号否则：链接器错误;2 如果有一个强符号和多个弱符号，选强符号；3 如果有多个弱符号，选择任意一个 **（注意字节）**

避免：用 static：定义全局变量，就初始化它；引用外部全局符号使用 extern

全局符号：当前模块定义，并能被其他模块引用的符号 – 强符号：函数和已初始化的全局变量，不可重复定义 – 弱符号：未初始化的全局变量，可重复或被强符号覆盖，如果有多个弱符号无强符号则任选一个，只在链接前存在，现代编译器默认禁止隐式弱符号 局部符号(Local)：仅在当前文件可见，static修饰，并非局部变量 外部符号：由其他模块定义被当前模块引用的全局符号，extern修饰，**静态**链接后消失

int g=10;//强符号//void func(){} int main （）也是
int a；double z；//弱符号

6 静态库(.a)将一组相关的可重定位目标模块.o 打包成一个单独文件

解析过程:扫描 E（要合并文件）、U（未解析符号）、D（已定义符号）

链接：保存和运行的可执行文件存在重复；库 更新了程序要重新编译

7 共享库(.so)执行文件第一次加载运行，动态链接就出现；也可以再程序启动后进行；内存特点：物理内存中只保留一份 .text 被所有进程共享

实现方法：加载时链接由 ld-linux.so 在程序启动时处理；运行时动态链接：通 过 dlopen() / dlsym （） 接口在程序执行中加载，通过dlclose**卸载**

8 库打桩（待定）

第八章异常控制流 1.EFC:普通用跳转调用实现根据程序内部变量变化反应异常:系统需要对外部状态如 Ctrl+C 做出反应，这些不是程序内部能预料

linux 系统调用的**功能号**:区分不同系统服务(read,fork 的编号)**异常号**:操作系统中断向量表的索引，**所有系统调用通过同一个异常号；**

2 异常:为响应某个事件将控制权转移到操作系统内核中的情况 **处理流程：**

发生事件→CPU 通过异常号索引异常表→跳转到异常处理程序→

①返回当前指令重新执行②返回下一条指令③终止程序 四种异常：

1 **中断**(异步异常):处理器外部 IO 设备引起,**返回下一条指令**(ctrlC,数据到达)

2 **陷阱**:有意执行指令，**返回下一条**(syscall(open,断点，用户程序和内核接口)

3 **故障**:潜在可修复错误，**返回当前**(缺页异常，非法引用，浮点异常除 0)

4 **终止**:不可修复致命错误，**不返回终止**(非法指令，奇偶校验，硬件)

3 进程 **逻辑控制流**(似乎独占 CPU)私有地址空间(似乎独占内存)上下文：包括通用寄存器值；程序计数器；用户栈和内核栈；页表；内核数据结构；**并发进程**:两个进程在时间上有重叠，否则顺序 用户态内核态？

4 进程控制:1.fork()调用一次返回两次，在父进程返回子进程 PID，子返回 0

子进程复制父进程的所有状态，但修改子进程的变量不会影响父进程

创建时父子进程相同虚拟→物理地址，写时复制物理地址不一致

2.回收子进程:保持终止，被父回收，若父先退，被 init 进程(PID1)收养

pid_t getpid(void)返回当前进程 PID pid_t getppid(void)返回父进程 PID

Void exit(int status)终止进程：exit 不返回到下一行，waitpid()回收(阻塞)

int execve(const char *filename, char *argv[], char *envp[])文件路径参数列表环境变量列表，调用一次不返回:成功 不创建新进程，覆盖旧的(PID 不变)

5**信号**处理过程:shell 最高层的软件形式的异常，允许进程和内核中断其他进程

发送:内核更新目的进程上下文中的某个状态，发送一个信号给目的进程

接受:目的进程被内核强迫以某种方式对信号做出反应(忽略，终止，捕获)

待处理信号和阻塞信号:发出未接受，一类至多有一个 **kill -n PID 发送信号**

SIGINT 终止(键盘中断)SIGKILL 终止(杀死程序)SIGSEGV 终止转储内存(内存引用段错误)SIGALRM 终止(定时器)SIGCHLD 忽略(子进程停止终止)SIGFPE(/0)

6 信号处理的安全性:1 处理程序尽可能简单；2 只调用异步信号安全函数：记住 printf, malloc, exit 是不安全的！ 安全函数包括_exit, write, waitpid；3 保存和恢复 errno；4 共享变量用 volatile sig_atomic_t 声明

有个奇怪的题：在编写 Shell 时，父进程 fork 出子进程，父进程需要把子进程加入作业列表，而子进程结束会触发 SIGCHLD 处理程序如果子进程运行极快，在父进程还没执行 addjob 之前，SIGCHLD 处理程序就运行了并尝试 deletejob。这会导致作业列表中多了一个永远无法删除的“死项”在 fork 之前，必须先阻塞 SIGCHLD，确保父进程先完成 addjob，然后再解除阻塞。

7 非本地跳转 setjmp(调 1 返 2)和 longjmp(调 1 不返但跳回最近 setjmp)

第九章虚拟内存 1 地址空间 **Inter x86CPU 生成逻辑地址→段式管理(MMU)→线性地址(=虚拟地址)→页式管理(MMU)→物理地址**

逻辑地址空间：段地址(段标识符 CS/DS/ES)+偏移地址来表示（实模式下是 16 位段寄存器+ 16 位偏移；保护模式下是 16 位段寄存器 + 32/64 位偏移）

线性地址空间：非负整数地址的有序集合 linux 线性地址~虚拟地址

虚拟地址空间：N = 2^n 个虚拟地址的集合===线性地址空间 N>>M

物理地址空间：内存芯片真实地址 M = 2^m 个物理地址的集合

指令集架构(ISA)定义.***逻辑地址***的计算规则，**段式管理机构**实现 ISA 计算(仅硬件) ***虚拟→物理地址***：由操作系统(页表)+MMU(硬件)协同

逻辑地址：汇编代码中的偏移[如 8(%rsp)]

虚拟地址:CPU 发出的地址，程序页汇编的，链接器操作的(符号解析重定位后，符号被分配到确定的虚拟地址)指针变量的值,用来访问页表的地址

物理地址:访问内存的，MMU 的输出，页表项 PTE 存储的内容，用来访问 Cache 和 DRAM 的(L1 Cache 通常是物理寻址)

段寄存器（16 位）用于存放段选择符：CS(代码段)：SS(栈段)：DS(数据段)：全局静态数据区所在段其他 3 段寄存器 ES、GS 和 FS 可指向任意数据段

被选中的段描述符先被送至描述符 cache，每次从描述符 cache 中取 32 位段基址，与 32 位段内偏移量（有效地址）相加得到线性地址

2 虚拟内存 VM

磁盘是虚拟内存的储存→物理(DRAM)

是磁盘的缓存→Cache(SRAM)是内存物理内存的缓存→CPU

1.内存可视作为磁盘的缓存，程序使用虚拟地址访问 内存，操作系统负责将虚拟地址映射到物理地址（DRAM 或磁盘交换空间）**2.**虚拟内存用于：缓存，内存保护，内存管理 **3.**已申请的内存为懒加载，第一次访问触发缺页异 常；未申请的内存访问触发段错误(保护异常) **4.**虚拟地址到物理地址映射的建立由内核缺页处理程 序完成

9. 虚拟内存

内核	内核代码 & 数据
进程虚拟内存	进程相关数据结构
	环境变量
	命令行参数 argv/c
	用户栈 ↓ %rsp 减小
	十 共享库内存映射
	↑ brk 增大 运行时堆
	数据段.bss
	数据段.data
	代码段.text 等

DRAM:DRAM 缓存的组织结构完全是由巨大的不命中开销驱动的

缓存单元（虚拟页面）：(未分配，已缓存，未缓存三种状态)

存放位置： 物理页/页帧 DRAM 缓存为全相联，任何虚拟页都可放置在任何物理页中;总是使用写回，不是直写;更复杂精密的替换算法

2 缓存数据结构：页表，存放在 DRAM 的一个 PTE 中

PTE 构成:每个条目包含一个有效位(0 则缺页异常)+物理页号或磁盘地址

缺页异常→选择牺牲页→导致缺页的指令重新启动→页命中

当有不命中发生时才换入页面的这种策略 称为页面调度

虚拟内存系统能有效工作的前提？ 虽然 DRAM 缓存未命中代价很高，但因为程序具有良好的局部性，活跃的虚拟页通常只占一小部分（称为**工作集**，工作集大小>DRAM 会**抖动**，页面不断换进换出导致性能崩溃）

为什么 DRAM 缓存不采用 SRAM 缓存那样的组相联结构？ 为了最大程度降低缺页率，因为磁盘访问延迟极高 必须用上方画线句

3 地址翻译 基本参数 N = 2^n：虚拟地址空间中的地址数量

M = 2^m：物理地址空间中的地址数量 P = 2^p ：页的大小 (bytes)

PTE:页表项 数量为 2^n-p **TLB:**快表，是页表的缓存

TLBI: TLB 索引 **TLBT**: TLB 标记 **VPO**: 虚拟页面偏移量 **VPN:**虚拟页号

PPO: 物理页面偏移量 **PPN**: 物理页号

①把虚拟地址化成 2 进制，根据题设给定是几位地址就截断成几位

②拆解 **虚拟地址(VA)**:由 n-p 位 VPN 和低 p 位 VPO 构成

虚拟地址=VPN+VPO 而 VPN= TLBT+ TLBI

TLB 索引：TLBI=log2(TLB 组数) 组数=行数/路数(见 Cache 组相联)

③先查 TLB 根据 TLBI 找到 TLB 的组(0x03)，看该组内的几个条目里（a 路组相联则是 a 个），TLBT=标记位且有效位为 1 的条目是否存在，存在则记录 PPN，不存在则不命中，不命中跳转到四

④查页表（如果三失败）

页表按 VPN 索引，查页表中 VPN 对应的 PTE，PPN，有效位=1→不缺页 得到 PPN

⑤拼接物理地址，再转 16 进制（地址的书写形式是 16 进制）

物理地址(PA):由 PPN(页表查询)和 PPO(PPO=VPO)组成

⑥用物理地址查 L1 d-cache，判断缓存命中，提取返回字节

CO（字节偏移）= log2（每个块的字节数）

CI（缓存组索引）=log2（组数） **CT**（缓存标记）：剩下的位

PA = CT(PPN) +CI+CO(PPO) 这里拆分是 2 进制

页面命中：CPU 发送虚拟地址(VA)→MMU 生成页表条目地址(PTEA)并从内存读取 PTE//→MMU 将物理地址(PA)给高速缓存/主存→处理器(CPU)

缺页：//→**MMU 发现 valid=0 抛出异常**→缺页处理程序确定物理内存中的牺牲页→更新内存 PTE→返回原来进程→//

*Intel CPU 访问一个内存数据，可能会访问几次存储器？*最多 2 次最少 0 次

0 次:Cache 命中，TLB 命中，直接读取 Cache

2 次:Cache 未命中访问物理内存读取数据 1 次，写入 cache；TLB 未命中访问物理内存查页表一次，完成地址转换

4 为什么需要 TLB:查页表要看内存大惯。TLB 是 MMU 内部的一个小缓存流程：CPU 发出 VA→MMU 拆出 TLBT/TLBI→TLB Hit 则直接出 PPN

5 多级页表：一级页表的 PTE→ 下一级页表→最后一级包含 PPN

VPN 被拆分成多个部分(页目录索引 VPN1，页表索引 VPN2)

Intel X86-64 的现代桌面 CPU，采用（4）级页表

*为什么可以节省存储空间？*1 按需创建如果一级页表中的某个条目是空的，那它指向的二级页表就完全不需要存在。2 只有一级页表需要常驻内存

1.异常处理子程序是所有进程共享的内核代码。

2.信号处理子程序是每个进程独享的用户代码。

6 内存映射(mmap)：虚拟地址空间与磁盘文件的映射，通过将虚拟内存区域与磁盘上的对象关联起来以初始化这个虚拟内存区域的内容.

虚拟内存区域可以映射的对象:**磁盘上的普通文件** (e.g.可执行目标文件)

匿名文件（内核创建，**请求二进制零**，开始不需要读).bss 栈堆采用此机制

共享对象：多个进程映射到同一物理内存***私有对象***：使用写时复制(COW)

解释：读时,多进程的虚拟页映射到同一份物理页（共享对象）；写时，触发 COW，分配新的物理页，使写操作仅影响自己的私有物理页（私有对象）

7 COW 原理：**fork()**创建进程时，子进程**拷贝父进程的页表，但物理页面被标记为 只读**；**触发**：只要有进程尝试**写**页面，就会触发保护故障，内核会为该进程**创建一个新**的物理副本 **优点**:极大节省内存，避免不必要拷贝

8 execve 函数 **销毁旧页表，创建全新页表映射**，完全替代当前进程的用户态内存空间(**保留 PID**,内核态) **execve 后的代码永不执行**

私有区域映射(COW)：.text 和.data(已初始化)映射到可执行文件中的对应节.bss(未被初始化的数据)和栈映射到匿名文件（初始为 0） 堆 Heap 和栈 Stack 在 execve 初始化时映射的是“匿名文件”，即请求二进制零的内存。

共享区域映射：共享库(libc.so)由动态链接器映射到共享区域

主要和 fork 做区分 fork 物理内存与父进程共享物理页（写时复制）

execve 撤销旧共享，映射新文件的物理缓存页

9 程序加载的完整流程

创建进程：用户态出发，shell 调用 fork()调用子进程 (./hello)

①子进程调用 execve，shell 的子进程从用户态陷入**内核态**;内核先校验 hello 的合法性（ELF 格式、执行权限）

②销毁当前进程的旧用户空间，如上

③**解析可执行文件的头部**（ELF Header），获取各个段的信息和入口地址

④创建虚拟内存区域(虚拟地址映射)，**先不分配物理内存**，**此时创建 PTE PTE 的状态置 valid=0,物理页号 PPN 为空**

映射目标：.text/.data 指向磁盘文件，.bss/堆/栈指向匿名文件

⑤初始化进程上下文 内核把 PC 设置为 ELF 的入口地址(_start)

把 argv、envp 拷贝到栈的虚拟地址空间：初始化进程的内核态数据

⑥返回**用户态**，首次执行指令，CPU 访问_start→缺页中断

⑦处理缺页中断，按需分配物理内存，**缺页异常处理子程序**写入 PPN

程序执行：⑦缺页中断触发开始 然后开始处理执行(陷入内核态)

⑧分配空闲物理页:.text/.data 从磁盘加载数据，.bss/堆/栈则初始化为 0；

⑨将分配的 PPN 写入 PTE，置位有效位

⑩中断返回，CPU 重新执行_start 指令，此时 MMU 页命中

若程序调用 printf 等库函数，会触发 libc.so 动态库的虚拟映射和缺页中断

最终执行完 hello 的逻辑，子进程退出，Shell 父进程回收子进程资源

malloc 函数功能（memory allocation 内存分配）void* malloc(siaze_t size)

输入：想要的字节数 输出：成果返回这块内存的首地址(指针)

如果失败(内存不足)：返回 null

申请到的内存内容是随机的，必须手动调用 free()释放，否则造成内存泄漏

malloc 申请的内存在堆 Heap 区域

当堆空间不够时，malloc 会通过执行系统调用来让内核增加堆的大小

参数表(argv)的结构 本身是一个指针，指向指针数组

以 NULL 结尾：指针数组的最后一个元素必须是 0x0000000000000000

之后是地址指向的实际内容 字符串 1（程序名）00 结尾

字符串 2 （参数 1） 00 结尾 字符串 3 （参数 2） 00 结尾

指令类型 常见指令 + 操作数	格式化说明模板
栈帧操作 push %rbp pop %rbx	保存调用者的[寄存器名]到栈中(保护现场)
mov %rsp, %rbp	建立当前栈帧，使 rbp 指向当前栈顶
sub $0xN, %rsp	为局部变量分配 N 字节栈空间
leaveq	恢复栈帧（等效于 mov %rbp, %rsp; pop %rbp）
参数传递 lea offset(%rbp), %rdi	将栈中[变量/缓冲区]首地址存入 rdi，作为第 1 个参数
mov label(%rip), %rsi	将全局变量/地址存入 rsi，作为第 2 个参数
函数调用 call <函数名>	调用指定函数
空间释放 add $0xN, %rsp	释放局部变量占用的栈空间
恢复现场 pop %rbx	从栈中恢复原 rbx 值
函数返回 retq	从栈中弹出返回地址并跳转（函数返回）

漏洞必现位置：

不安全函数调用 → strcpy、gets、scanf("%s")、sprintf 等无长度检查的函数栈空间分配不足 → sub 指令立即< 实际需要复制的数据长度

返回地址可被覆盖 → 缓冲区起始地址 + 分配大小 ≥ 返回地址偏移量（通常为缓冲区大小 + 8(rbp) + 8(返回地址)）

防范措施:

编译时：加-fstack-protector（金丝雀）

编码时：用 strncpy、fgets 等安全函数，并检查长度

系统级：开启 ASLR、DEP/NX（栈不可执行）

PPN长度12，VPN总长36，VPN1-4各自长度均为9

VPO (虚拟页偏移)：由页面大小决定（i7 通常为 4KB，即 12 位）。

TLBI (TLB 索引)：由 TLB 的组数决定。例如图中 L2 TLB 有 512 个条目，

4 路组相联，则组数 S = 512 / 4 = 128，所以 TLBI 占 7 位 (2^7 = 128$) 。TLBT (TLB 标记)：剩余的高位 VPN 部分

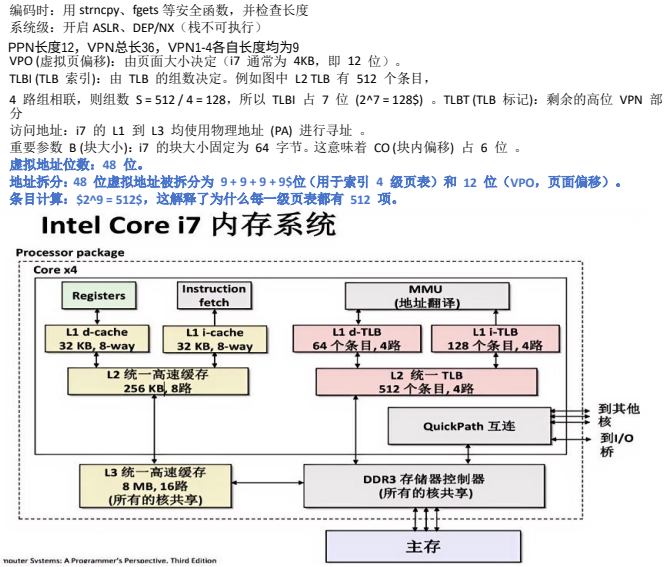
访问地址：i7 的 L1 到 L3 均使用物理地址 (PA) 进行寻址 。

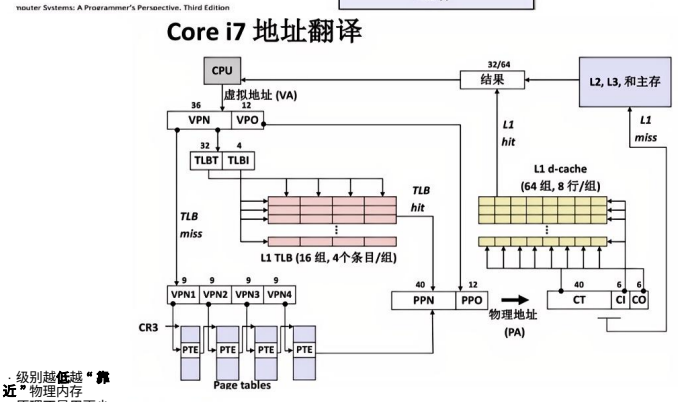
重要参数 B (块大小)：i7 的块大小固定为 64 字节。这意味着 CO (块内偏移) 占 6 位 。

虚拟地址位数：48 位。

地址拆分：48 位虚拟地址被拆分为 9 + 9 + 9 + 9$位(用于索引 4 级页表)和 12 位（VPO，页面偏移）。

条目计算：$2^9 = 512$，这解释了为什么每一级页表都有 512 项。





物理页表4KB对齐

级别越**低**越**“靠近”**物理内存

原理不是用更少页表管理更多 内存，而是稀疏分配节省内存

fork()实现，创建当前进程副本

1.创建当前进程的mm_struct,vm_area_struct和页表的副本，为子进程分配 PID

2.将父子进程的每个页都标记为只读 **3.**将父子进程的每个vm_area_struct都标记为私有的 COW **4.**在子进程中返回时，子进程拥有与 fork 调用时父进程相同的虚拟内存 **5.**子进程之后的写操作通过 COW 创建新页

execve()实现，替换当前进程

1.删除已经存在的用户区域vm_area_struct **2.映射私有区域**创建新的vm_area_struct，私有 COW，将目标文件的.text，.data映射到代码和数据区，将.bss和堆栈映射到匿名文件；堆栈初始长度 0 **3.映射共享区域**用动态链接器将共享对象映射到新进程的共享区域，并对重定位的代码和数据进行重定位 **4.**在栈中建立程序参数和环境变量 **5.设置程序计数器**设置 PC 指向程序入口_start，返回用户态运行新程序 **6.**运行中通过缺页中断处理程序按需加载代码或数据页，更新页表 **7.**进程终止后向父进程发送 SIGCHLD信号 ，（指令和数据在程序第一条指令开始执行时才被读入主存储器）

在终端中的命令行运行显示“ Hello World ”的执行程序 hello，结合进程创建、加载、缺页中断、到存储访问（虚存）……等等，论述 hello 是怎么一步步执行的。 答：进程创建：fork. 程序加载执行：execve、动态链接、mmap 等虚拟地址空间，执行指令：代码段缺页中断，恢复运行。(TLB、Cache、RAM) 访问数据：数据段缺页中断，恢复运行。显示输出：C 标准函数调用 printf、堆栈段缺页、系统调用 write，系统级 IO，显示 RAM 等。进程终止：释放。

什么是共享库（动态连接库）？简述动态链接的实现方法。 答：共享库（动态连接库）是一个.so 的目标模块（elf 文件），在运行或加载时，由动态链接器程序加载到任意的内存地址，并和一个和内存中的程序（如当前可执行目标文件）动态完全连接为一个可执行程序。使用它可节省内存与硬盘空间，方便软件的更新升级。如标准 C 库 libc.so 。加载时动态 链接：应用程序第一次加载和运行时，通过 ld-linux.so 动态链接器重定位动态库的代码和数据到某个内存段，再重定位当前应用程序中对共享库定义的符号的引用，然后将控制传递给应用程序（此后共享库位置固定了并不变）。运行时动态链接：在程序执行过程中，通过 dlopen/dlsym 函数加载和连接共享库，实现符号重定位，通过 dlclose 卸载动态库。

Linux 系统中，Shell 是一个交互型应用级程序，代表用户运行其他程序(是命令行解释器)，以用户态方式运行的终端进程)。 其基本功能是解释并运行用户的指令，置复如下处理过程：

(1)终端进程读取用户由键盘输入的命令行。 (2)分析命令行字符串，获取命令行参数，并构造 传递给 execve 的 argv 向量 (3)检查第一个(即 argv[0])命令行参数是否是一个内置的shell命令 (4)如果不是内部命令，调用 fork( )创建新进程/子进程 (5)在子进程中，用步骤2获取的参数，调用 execve( )执行指定程序。 (6)如果用户没要求后台运行(命令末尾没有&号) 否则 shell 使用 waitpid （或 wait…)等待作业终止后返回。 (7)如果用户要求后台运行 (如果命令末尾有 & 号) ，则 shell 返回；

TinyShell

” shell 定义：命令行解释器，提供用户与操作系统交互的接口，一个交互型的应用级程序，它代表用户运行其他程序；泰山服务器 ARM 架构上char为无符号类型，需要改为signed char强制指定，否则getopt返回-1时会被解析为255

运行命令的流程

1.判断是否为内置命令，若是则直接执行2.否则fork()创建子进程3.在子进程中用execve()加载执行程序4.父进程用waitpid()等待子进程结束并回收5.子进程调用exit()()退出并向 shell 发送SIGCHLD信号6.父进程捕获SIGCHLD信号并回收子进程7.shell 回到命令提示符，等待输入

fork的子进程与其父进程同名的全局变量对应同一物理地址？fork时子进程获得父进程数据空间、堆和栈的复制，所以变量的地址（当然是虚拟地址）也是一样的。每个进程都有自己的虚拟地址空间，不同进程的相同的虚拟地址显然可以对应不同的物理地址。因此地址相同（虚拟地址）而值不同没什么奇怪。fork子进程完全复制父进程的栈空间，也复制了页表，但没有复制物理页面，所以这时虚拟地址相同，物理地址也相同，但是会把父子共享的页面标记为“只读”，如果父子进程一直对这个页面是同一个页面，如道其中任何一个进程要对共享的页面“写操作”，这时内核会复制一个物理页面给这个进程使用，同时修改页表。而把原来的只读页面标记为“可写”，留给另外一个进程使用。假定父进程malloc的指针指向0x12345678, fork 后，子进程中的指针也是指向0x12345678，但是这两个地址都是虚拟内存地址（virtual memory），经过内存地址转换后所对应的 物理地址是不一样的。所以两个进城中的这两个地址相互之间没有任何关系。就是说：子进程继承父进程的全局变量。子进程创建以后，可以读取原来父进程的全局变量的值，但是创建以后，子进程修改了变量，或者是父进程修改了变量值，互相都不影响了。无论是什么类型的变量，fork后父子进程中都是一样的，但两者之间没有关系，任何一个进程修改变量后，在另一个进程中都不能知道，更不能访问另一个进程中的变量，也不会对应相同的物理地址了。**信号与异常的区别与联系**异常就是控制流中的突变，用来响应处理器状态的某些变化；信号是一条小消息，通知进程系统中发生了一个某种类型的时间。区别信号是软件上的概念，指的是由内核产生，并由用户态程序响应的事件。异常是硬件上的概念，指的是 CPU 因执行某些指令而产生的事件，比如常见的除零错误。另外，相对于中断，异常是同步的，中断是异步的；异常是由执行指令产生的，中断则是由硬件产生的。信号是异步的进程间通讯机制,是在软件层次上对中断机制的一种模拟。联系件异常也能产生信号，例如被零除、无效内存引用等。这些条件通常先由内核硬件检测到，然后通知内核。内核将决定产生什么样的信号。

获取反汇编的代码的指令：objdump -d 文件名

每个信号类型都有一个预定义的默认行为：进程终止；进程挂起直到被sigcont重启；忽略此信号。

x = (x OP d[i]) OP d[i+1] 与x = x OP (d[i] OP d[i+1])哪一个并行性好？x = x OP (d[i] OP d[i+1]) 的并行性更好,下一个循环的操作可以早一些开始，对之前的数据没有依赖。**Cache访问不命中的种类？**冷不命中（强制不命中）：当缓存为空时，对任何数据的请求都会不命中;冲突不命中：当缓存足够大，但是被引用的对象都映射到同一缓存块中，此种不命中称为冲突不命中;容量不命中：当工作集(working set)的大小超过缓存的大小时，会发生容量不命中**MMU怎么分配一个新的页面（VM/PM/PTE）**当操作系统分配一个新的虚拟内存页时，调用malloc的结果。分配过程即为在磁盘上创建空间并更新PTE中对应的页表，使它指向磁盘上这个新创建的页面

信号 使用 kill -? <pid> 发送 SIGINT(2): 中断，Ctrl+C SIGKILL(9): 强制终止，无法捕获和忽略 SIGSEGV(11): 无效内存访问，段错误 SIGCHLD(17): 子进程状态改变通知父进程 SIGFPE(8): 浮点异常，但整数除 0 也有 SIGALRM(14): 定时器到期 SIGSTOP(19): 暂停进程，无法捕获和忽略，Ctrl+Z SIGCONT(18): 继续执行暂停的进程 SIGTERM(15): 默认行为终止进程，可捕获和忽略